



Communicate with hardware

@ u-boot level

Communicate with Hardware (Memory Mapped I/O)

Read CONTROL STATUS Register:

Control Module Base Address is 0x44E1.0000 ; CONTROL STATUS Register offset address is 0x40

PA of CONTROL STATUS REG = Base Address of control module + offset address of control status register

```
= 0x44E10000
+      40
```

0x 44E10040

Physical Address = Base Address + Offset Address

Read control status register using md command:

=> md <address> [offset]

=> md 0x44E10040 (read word of data)

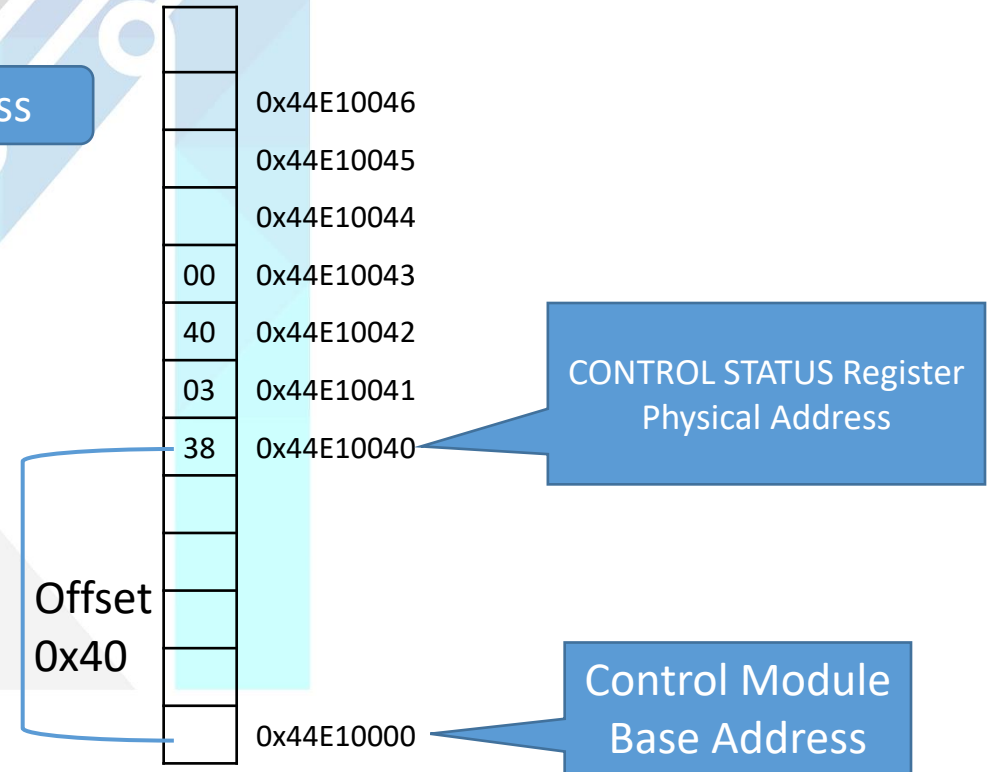
=> md.b 0x44E10040 (read byte of data)

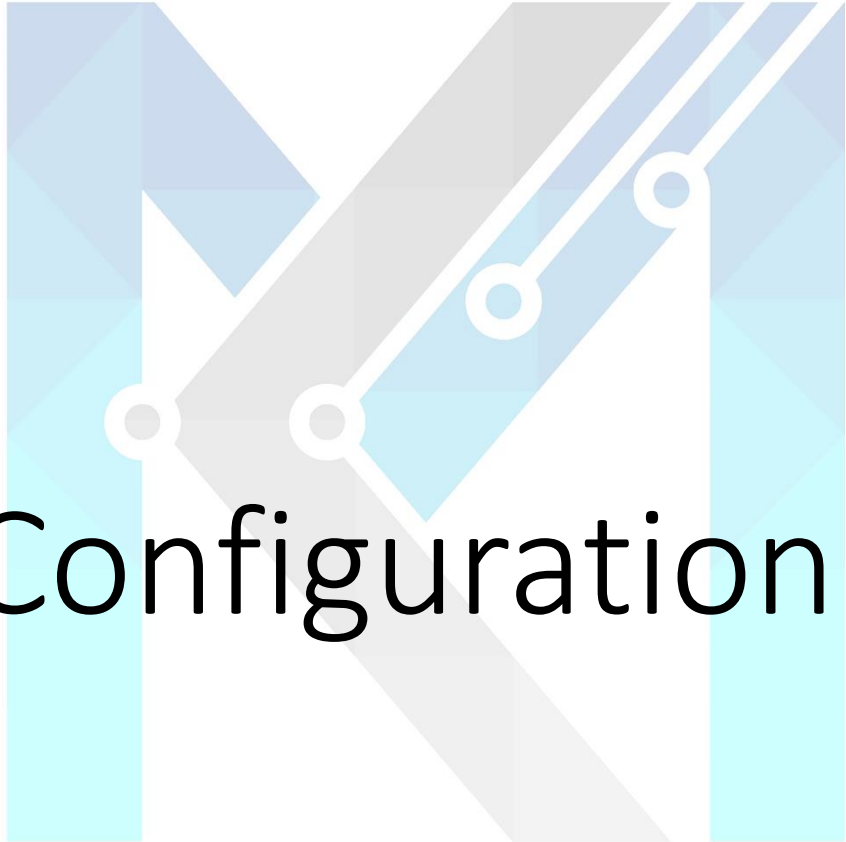
=> md 0x44E10040

44e10040: 00400338 00000000 00000000 00000000

SYS Boot Switch OFF – 11100

ON – 11000



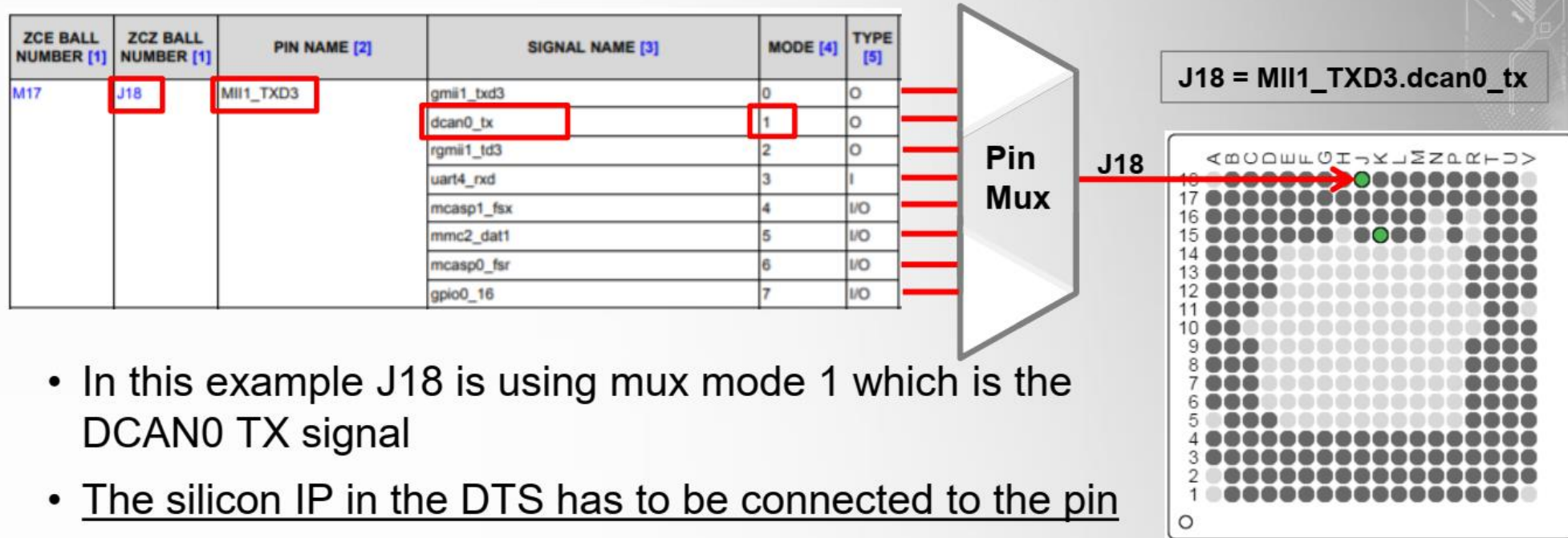


Mux/Pad Configuration

@ u-boot level

For SOC that use pin muxes - what is a pin mux?

- Most pins on the SOC have a mux must be set to enable a peripheral access
- Each pin name has several signal names that can accessed by a mux mode



- In this example J18 is using mux mode 1 which is the DCAN0 TX signal
- The silicon IP in the DTS has to be connected to the pin

Pad Control Registers: TRM – page no: 1369

9.3 Registers

9.3.1 CONTROL_MODULE Registers

Table 9-10 lists the memory-mapped registers for the CONTROL_MODULE. All other register offset addresses not listed in Table 9-10 should be considered as reserved locations and the register contents should not be modified.

Table 9-10. CONTROL_MODULE REGISTERS

Offset	Acronym	Register Description	Section
0h	control_revision		Section 9.3.1.1
4h	control_hwinfo		Section 9.3.1.2
10h	control_sysconfig		Section 9.3.1.3
40h	control_status		Section 9.3.1.4
110h	control_emif_sdram_config	control_emif_sdram_config Register (offset = 110h) [reset = 0h]	
428h	core_sldo_ctrl		Section 9.3.1.5
42Ch	mpu_sldo_ctrl		Section 9.3.1.6
444h	clk32kdivratio_ctrl		Section 9.3.1.7
448h	bandgap_ctrl		Section 9.3.1.8
44Ch	bandgap_trim		Section 9.3.1.9
458h	pll_clkinpulow_ctrl		Section 9.3.1.10
468h	mosc_ctrl		Section 9.3.1.11
470h	deepsleep_ctrl		Section 9.3.1.12

GPIO Registers: TRM – Page no: 4881

25.4 GPIO Registers

25.4.1 GPIO Registers

Table 25-5 lists the memory-mapped registers for the GPIO. All register offset addresses not listed in Table 25-5 should be considered as reserved locations and the register contents should not be modified.

Table 25-5. GPIO Registers

Offset	Acronym	Register Name	Section
0h	GPIO_REVISION		Section 25.4.1.1
10h	GPIO_SYSCONFIG		Section 25.4.1.2
20h	GPIO_EOI		Section 25.4.1.3
24h	GPIO_IRQSTATUS_RAW_0		Section 25.4.1.4
28h	GPIO_IRQSTATUS_RAW_1		Section 25.4.1.5
2Ch	GPIO_IRQSTATUS_0		Section 25.4.1.6
30h	GPIO_IRQSTATUS_1		Section 25.4.1.7
34h	GPIO_IRQSTATUS_SET_0		Section 25.4.1.8
38h	GPIO_IRQSTATUS_SET_1		Section 25.4.1.9
3Ch	GPIO_IRQSTATUS_CLR_0		Section 25.4.1.10
40h	GPIO_IRQSTATUS_CLR_1		Section 25.4.1.11
44h	GPIO_IRQWAKEN_0		Section 25.4.1.12
48h	GPIO_IRQWAKEN_1		Section 25.4.1.13
114h	GPIO_SYSSTATUS		Section 25.4.1.14
130h	GPIO_CTRL		Section 25.4.1.15
134h	GPIO_OE		Section 25.4.1.16
138h	GPIO_DATAIN		Section 25.4.1.17
13Ch	GPIO_DATAOUT		Section 25.4.1.18
140h	GPIO_LEVELDETECT0		Section 25.4.1.19

MuX/Pad Configuration

What is Mux Configuration:

Most of the pins in microcontroller consists of more than one functionality.

Using multiplexer to select the required functionality. That is called Mux/pad configuration

PA of LCD_DATA14 pad Reg = CM base add + offset

= 0x44E10000

+ 8D8

0x44E108D8 (PA of LCD_DATA14 MUX)

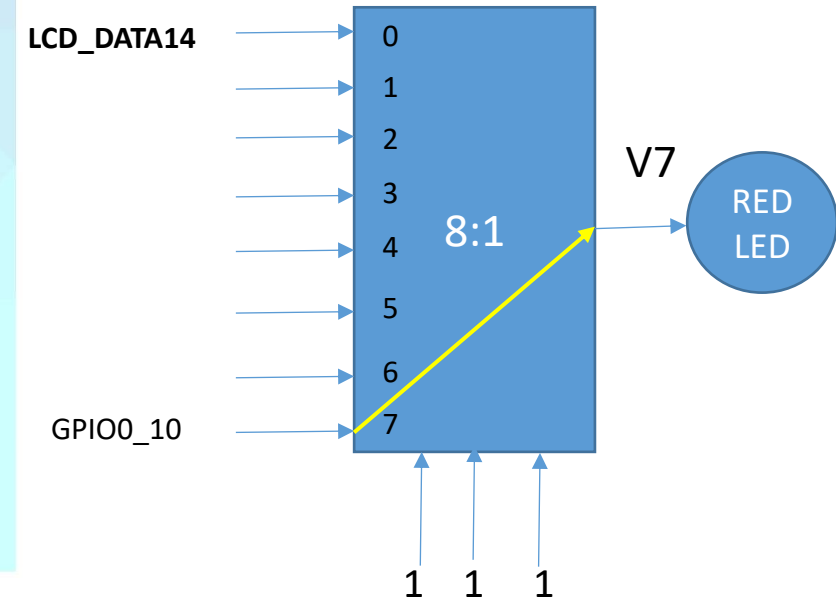
PA of GPIO_OE Register = 0x44E07000 + 0x134 = **0x44E07134**

PA of GPIO_DATAOUT Reg = 0x44E07000 + 0x13C = **0x44E0713C**

LCD_DATA14 Mux Register

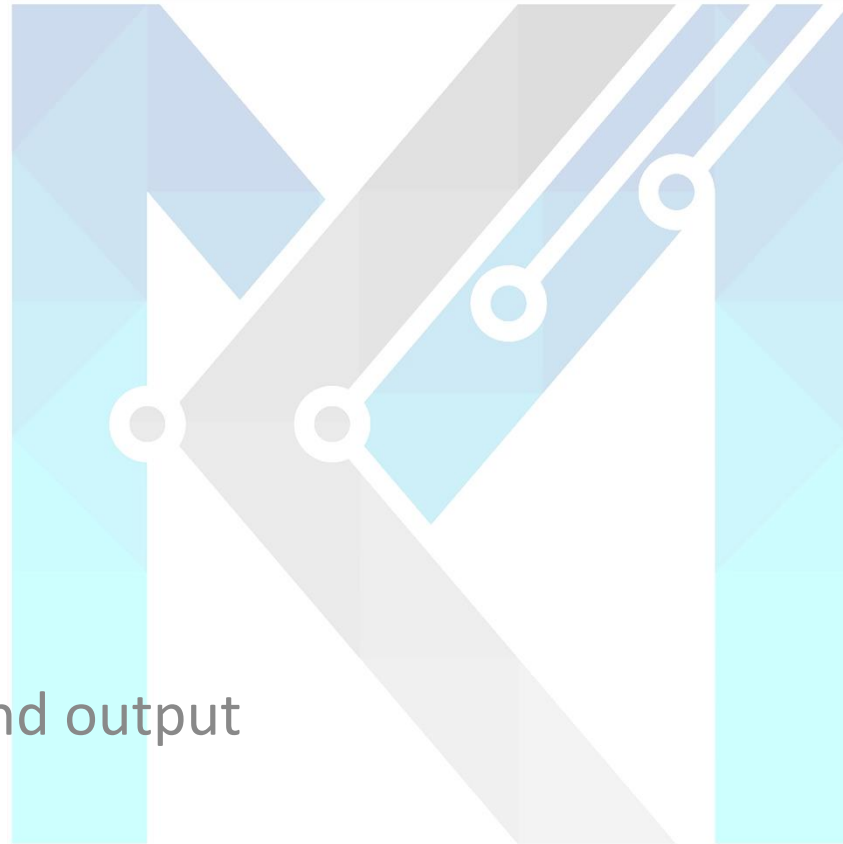
							1	1	1
--	--	--	--	--	--	--	---	---	---

MUX MODE

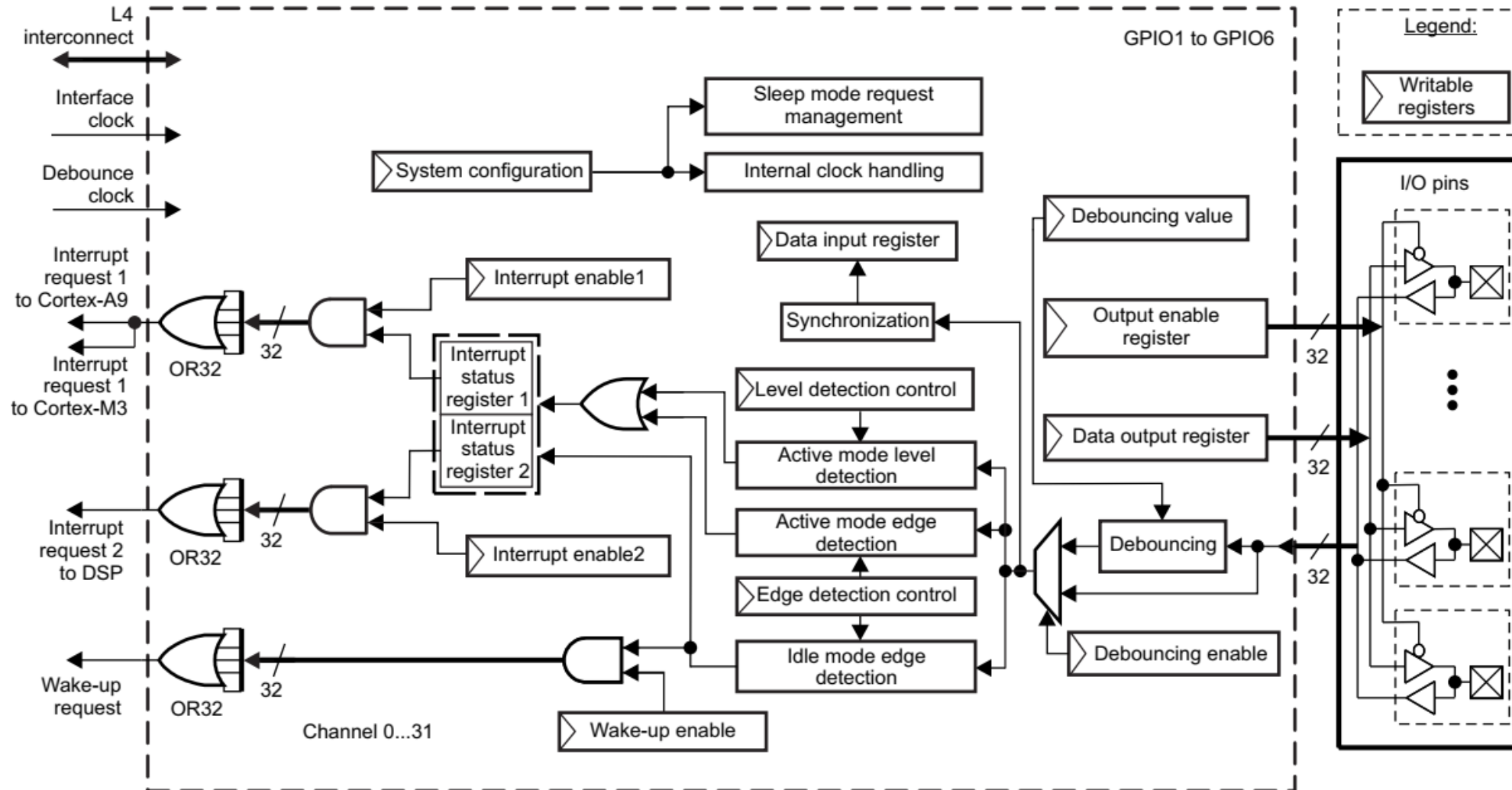


GPIO

General Purpose Input and output



AM3358 GPIO Controller



Control LED Device from u-boot prompt

To Control LED Device two things required:

1. LED Configuration (Activate):

Load mode bits 7 in LCD_DATA14 Pad register using mm command to **enable GPIO MUX**.

Clear 10th pin in GPIO_OE register using mm command to **set output direction**.

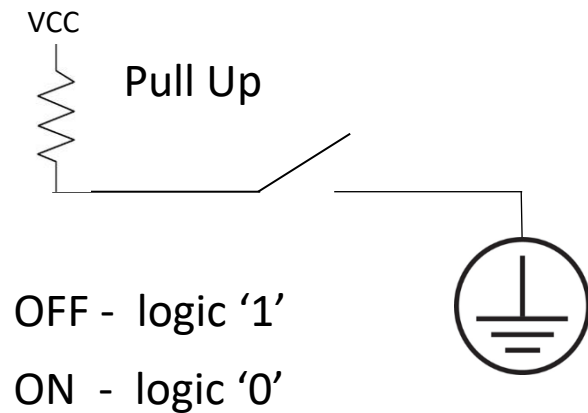
2. LED Operation:

Set 10th bit in GPIO_DATA register using mm command to LED ON.

Clear 10th bit in GPIO_DATA register using mm command to LED OFF.

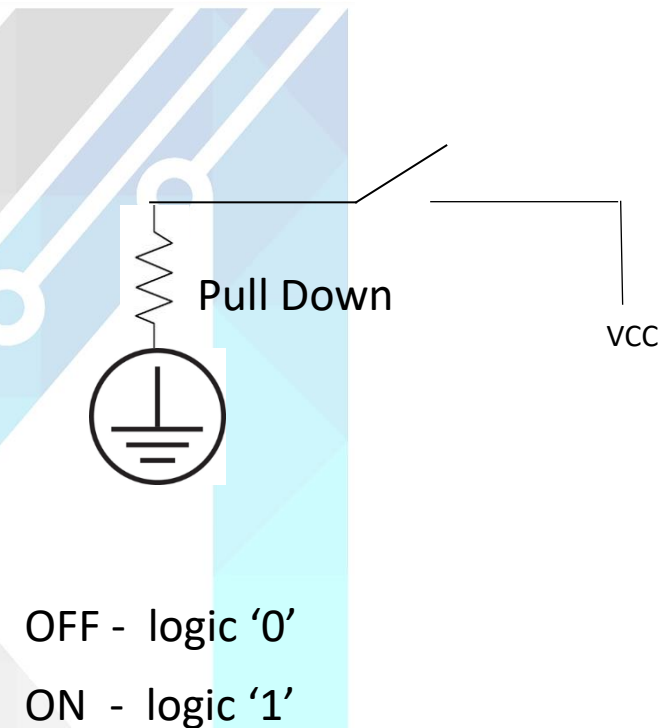
Types of switches

-ve level Switch



Pull up: Pin connected to supply
Pull down: pin connected to ground

+ve level Switch



GPIO0_11 configured as an either -ve level or + ve level switches
GPIO0_26 & GPIO0_27 configured as -ve level switches

Get switch value from u-boot prompt

To get switch value two things required:

1. Switch Configuration (Activate):

Load mode 3 bits in LCD_DATA15 Pad register using mm command to **enable GPIO MUX**.

Clear 3rd,4th bit in LCD_DATA15 Pad register to configured as PULL Down

Set 5th bit to configured as input.

Set 11th bit in GPIO_OE register using mm command to **set input direction**. (By default input direction).

2. Get switch value:

Read 11th bit in GPIO_DATAIN register using md command to get switch value..

Get switch value from u-boot prompt

PA of LCD_DATA15 pad Reg = CM base add + offset

= 0x44E10000

+ 8DC

0x44E108DC (PA of LCD_DATA15 MUX)

PA of GPIO_OE Register = 0x44E07000 + 0x134 = **0x44E07134**

PA of GPIO_DATAIN Reg = 0x44E07000 + 0x138 = **0x44E07138**

0x40000000

Pull Down: 0x27

ON: 0x0040 -> 0000 **0**000 0100 0000

OFF: 0x0840 -> 0000 **1**000 0100 0000

Pull Up: 0x37

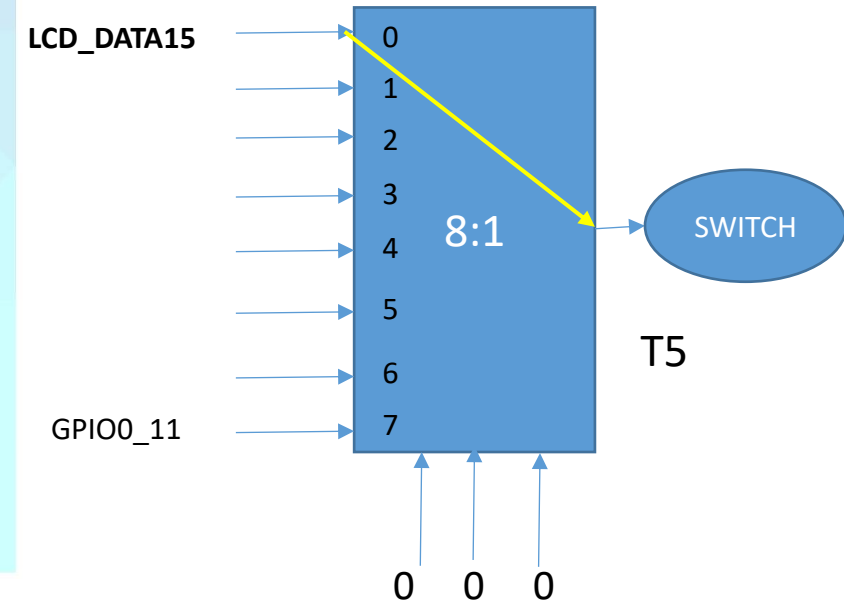
OFF: 0x0040 -> 0000 **0**000 0100 0000

ON: 0x0840 -> 0000 **1**000 0100 0000

LCD_DATA14 Mux Register

				0	0	0	0	0	0
--	--	--	--	---	---	---	---	---	---

MUX MODE



Read/write hardware registers

arch/arm/include/asm/arch-am33xx/mux_am33xx.h

__raw_writel()

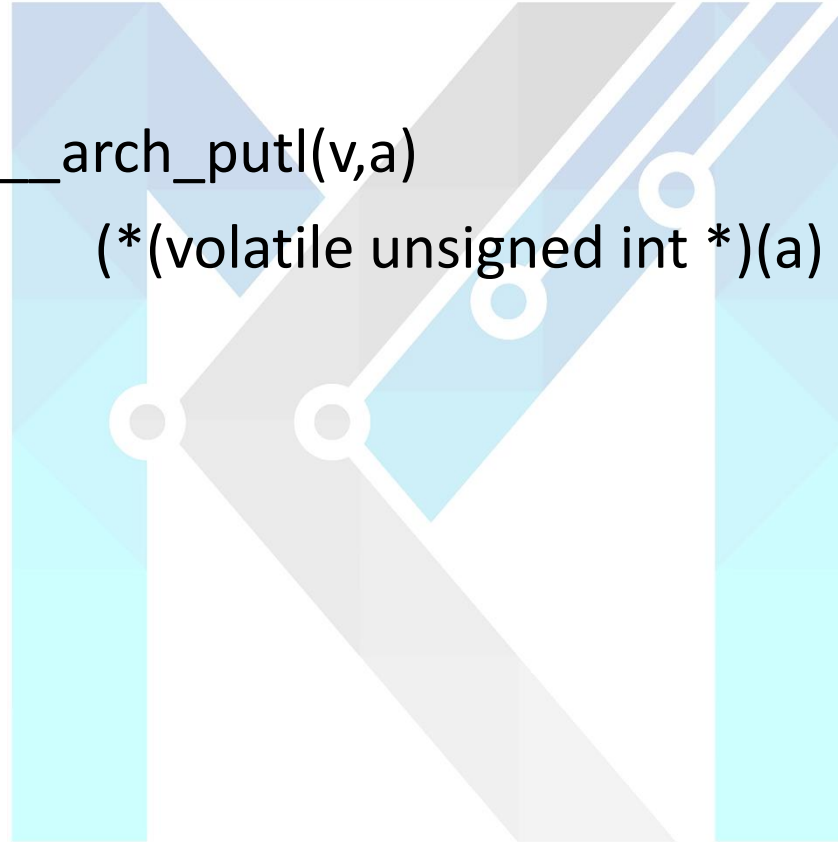
#define __raw_writel(v,a)

#define __arch_putl(v,a)

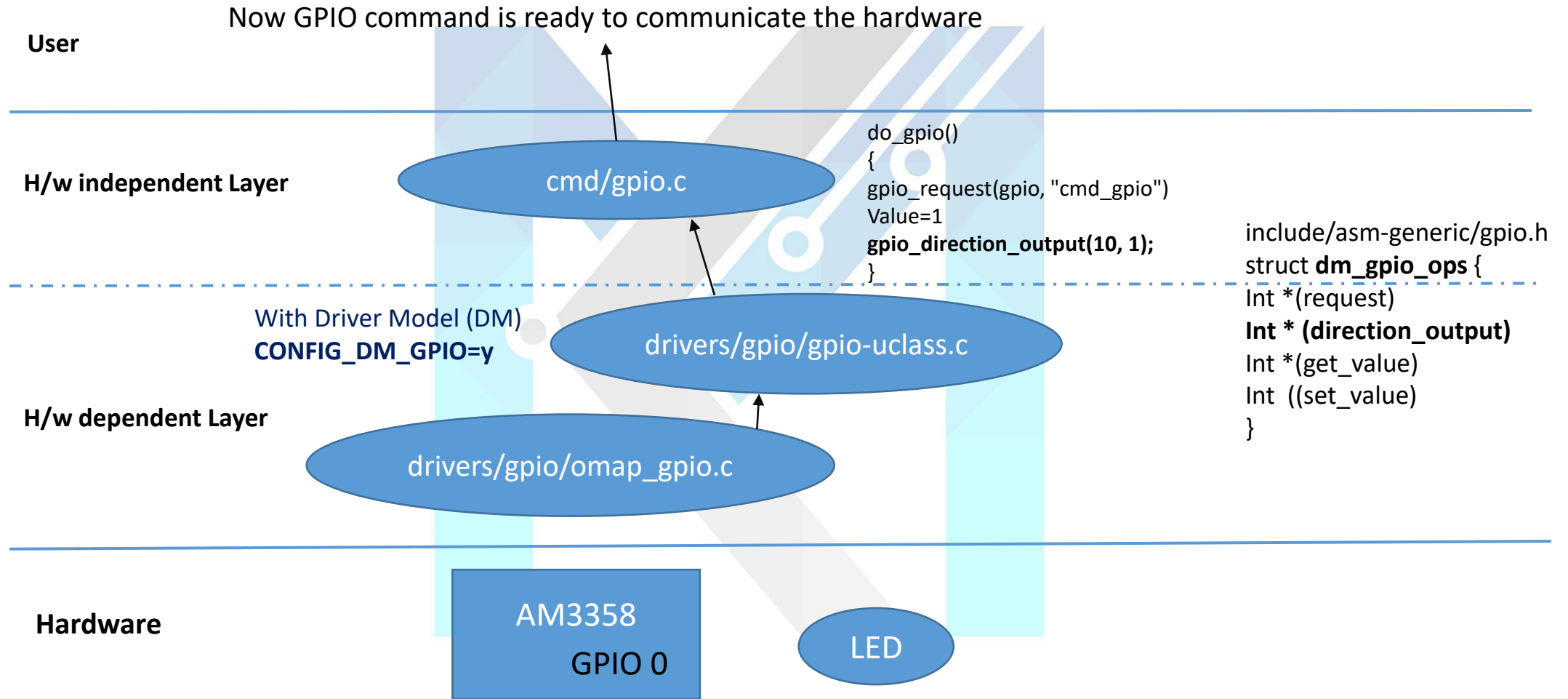
__arch_putl(v,a)

(*(volatile unsigned int *)(a) = (v))

arch/arm/include/asm/io.h



GPIO Initialization Framework in U-boot



GPIO Operation Framework in U-boot

